

이벤트 시스템 구현 — 안 4 (kind 분리 + chain)

선행 안: 이벤트 시스템 구현 — 안 3.md , 이벤트 시스템 — JSON 스키마 안 (A·B·C).md

스키마 안 B 의 변형을 채택. **이벤트 = 단일 책임 단위** — 한 이벤트는 한 kind 만 (대사 OR 효과 OR 영상 OR 선택 지). 복합 시퀀스는 **chain** (`next` 필드) 으로 작은 단위들을 연결해 표현. MVP = **dialogue** kind 만, 나머지 kind 와 chain 은 다음 차로.

§0. 확정 사항

- A. EventManager 오토로드 신설. BattleManager 와 동일 결 — `event_state` 는 이벤트 동안만 살아있고 비활성 시 `{}` .
- B. 이벤트 = JSON dict with `kind` 디스크리미네이터. `kind` 별로 페이로드 형태가 다름. 한 이벤트는 한 kind 만.
- C. kind 풀: `dialogue` / `effect` / `movie` / `choice` . MVP = **dialogue** 만 구현, 나머지는 스키마 자리만 명세하고 실 디스패처는 다음 차로.
- D. chain (`next` 필드) = 다음 차로. MVP 단계 이벤트는 단일 노드. `next` 도입 후엔 한 이벤트 종료 시 `next` 가 있으면 phase 안 빠지고 자동 전이.
- E. 영속 플래그 (`seen_events`) 는 `big_run_data` 안. RunManager 단일 추적점.
- F. Type 1 (mutating) = `RunManager._apply_*` 직 호출 단일 경로. `escape` (custom + fn) 없음.
- G. 시그널 계약 — `EventManager.event_resolved(result: Dictionary)` 한 번 emit. `BattleManager.battle_ended` 와 대칭.
- H. on/off 배타. 이벤트 활성화 중 외부 입력·다른 매니저 상태 변경 무시 — phase 게이팅 + UI lockout.

§1. 목표 / 검증 (MVP)

노드 type "event" 진입 → `EventManager.begin_event` → kind 디스패치 → `dialogue` 면 라인 진행 → 마지막 라인 advance → `event_resolved` → RunManager 가 phase 복귀.

- [] 노드 진입 시 `phase == "event"` 전이, `event_state` 활성화
- [] `dialogue` kind 이벤트 라인 N개 순차 진행
- [] 마지막 라인 advance → `event_resolved` 발화
- [] RunManager 가 수신해 `seen_events += 1` + phase 복귀
- [] `event_ui.gd` 삭제 시 시물 컴파일 에러 0건. 콘솔만으로 동일 작동.
- [] 비 `dialogue` kind (e.g. `effect`) 만나면 경고 + 즉시 `event_resolved` (스텝 동작)

§2. 모듈 경계

층	파일	역할
상수	game_data.gd	EVENTS + 노드 type "event"
시물 (이벤트 흐름)	event_manager.gd (신규 autoload)	진입·kind 디스패치·결과 emit
시물 (런 상태)	run_manager.gd	_apply_* 풀, seen_events, phase 전이
UI	event_ui.gd (신규)	dialogue 렌더 + advance 입력

의존 방향: GameData ← {RunManager, EventManager} ← event_ui . 역방향 0. EventManager → RunManager
호출: effect kind 도입 후 _apply_* 직호출 (MVP 시점엔 호출 없음).

§3. 자료구조

3-1. EventDefinition — 공통 필드

```
{
  "id": "string",
  "kind": "dialogue" | "effect" | "movie" | "choice",
  "trigger": {"type": "node_enter", "node_type": "event"},
  "once_per": "big_run",
  "weight": 10,
  "next": "string"
}
```

- id : 전역 유일 키. seen_events / chain 의 식별자.
- kind : 디스크리미네이터.
- trigger : 안 3 와 동일. 종류는 node_enter 만 우선.
- once_per : "big_run" 또는 생략. 회귀 통과 시 같은 풀에서 제외.
- weight : 동일 트리거 충돌 시 가중치 추천.
- next : (다음 차로) chain 다음 이벤트 id. null/생략 = 종료.

3-2. kind 별 페이로드

dialogue (MVP)

```
{
  "id": "intro_monologue",
  "kind": "dialogue",
```

```

    "trigger": {"type": "node_enter", "node_type": "event"},
    "once_per": "big_run",
    "weight": 10,
    "lines": [
      {"speaker": "heroine", "text": "여긴 어디지..."},
      {"speaker": "heroine", "text": "팔이 움직이지 않아."}
    ]
  }
}

```

- `lines` : 라인 배열. 각 entry = `{speaker: String, text: String}`.
- `speaker` : 화자 키 (현 단계는 표시명 그대로 사용해도 무방. CHARACTERS 레지스트리 도입은 다음 차로).

effect (다음 차로)

```

{
  "id": "...",
  "kind": "effect",
  "effects": [
    {"type": "body_boost", "amount": -5}
  ]
}

```

UI 입력 0초. `effects` 일괄 `_apply_*` → 즉시 종료.

movie (다음 차로)

```

{
  "id": "...",
  "kind": "movie",
  "movie": {"path": "res://cutscenes/intro.ogv", "skippable": true}
}

```

choice (다음 차로)

```

{
  "id": "...",
  "kind": "choice",
  "prompt": {"speaker": "heroine", "text": "어떻게 할까?"},
  "choices": [
    {"label": "응급 처치", "next": "treat_basic"},
    {"label": "포기", "next": "give_up"}
  ]
}

```

선택 시 그 가지의 `next` 로 chain 분기.

3-3. EventManager.event_state 스키마

```
{
  "event_id": "string",
  "kind": "string",
  "phase": "running" | "awaiting_input",
  "chain_root_id": "string",
  "line_idx": 0
}
```

- `chain_root_id` : chain 진행 중에도 `seen_events` 카운트는 root id 한 개로만 (다음 차로). MVP 단계엔 `= event_id`.
- `line_idx` : dialogue kind 의 현재 라인 인덱스. 다른 kind 는 미사용.
- 비활성 시 `event_state = {}`.

3-4. RunManager 자료구조 변경

- `big_run_data["seen_events"]`: Dictionary — `_new_big_run` 에서 `{}` 초기화.
- `phase` 풀에 "event" 추가.
- `move_to_node` 의 type 분기에 "event" 추가.

§4. 의존 DAG

GameData (leaf) — EVENTS / 노드 type "event"

↑

RunManager — 영속: `run_data`, `big_run_data` (`seen_events` 포함), `_apply_*`

EventManager — 일시: `event_state`. kind 디스패처.

|

└─ RunManager._apply_* (effect kind, 다음 차로)

└─ event_resolved (RunManager 가 수신)

`run_ui` ← RunManager.state_changed (screens dispatch by phase)

`event_ui` ← EventManager.event_state_changed

`event_ui` → EventManager.advance_line / select_choice

순환 0. 시물 → UI 0. `event_ui` → RunManager 직접 호출 0 (`run_ui` 가 visibility 담당).

§5. 단계 분할 (MVP)

Step 1 — EventManager 오토로드 + dialogue kind 흐름

game_data.gd : - EVENTS 상수 — dialogue kind 1~2개 등록. - TEST_MAP_GRAPH 의 노드 1개를 type "event" 로 변경 (또는 추가).

event_manager.gd (신규) : - event_state: Dictionary . 빈 dict = 비활성. -

begin_event(event_id: String, context: Dictionary) -> void - EVENTS lookup → kind 보고 분기 - dialogue: event_state = {event_id, kind: "dialogue", phase: "awaiting_input", chain_root_id: event_id, line_idx: 0} - 그 외 kind: push_warning , 즉시 _finalize_event (스텝) - event_state_changed.emit() - advance_line() -> void - dialogue 전용. line_idx += 1 . - 마지막 라인 통과 시 _finalize_event 호출. - event_state_changed.emit() - _finalize_event() -> void - event_resolved.emit({"event_id": chain_root_id}) - event_state = {} - _resolve_event_for_node(node: Dictionary) -> String - 트리거 + once_per 필터 + 가중치 추천. 결과 = event_id 또는 "" . - 시그널: event_state_changed , event_resolved(result: Dictionary) .

run_manager.gd : - _new_big_run 에 "seen_events": {} 추가. - move_to_node 의 type 분기에 "event" : var event_id := EventManager._resolve_event_for_node(target) if event_id != "" : run_data["phase"] = "event" EventManager.begin_event(event_id, {}) state_changed.emit() return true - _ready 에 EventManager.event_resolved.connect(_on_event_resolved) 추가. - _on_event_resolved(result: Dictionary) -> void : - big_run_data["seen_events"][result.event_id] = big_run_data["seen_events"].get(result.event_id, 0) + 1 - run_data["phase"] = "map" - state_changed.emit()

project.godot : EventManager 오토로드 등록.

LimboConsole : - event_advance — EventManager.advance_line() - show_event — event_state JSON 덤프.

검증: 노드 진입 → phase == "event" & event_state 활성화. event_advance 반복 → 라인 진행 → event_resolved 발화 → seen_events 증가 + phase = "map" 복귀.

Step 2 — UI (event_ui.gd) for dialogue

main.tscn : run_ui 자식으로 event_screen: Control + 스크립트.

event_ui.gd : - _ready :

EventManager.event_state_changed.connect(_on_event_state_changed) . -

_on_event_state_changed : - event_state.is_empty() → 화면 숨김. - event_state.kind == "dialogue" → 라인 렌더. - 그 외 → 일단 빈 화면 (다음 차로 분기 추가). - 라인 렌더:

EVENTS[event_id].lines[line_idx] 의 speaker / text 표시. 진행 버튼 (또는 화면 클릭) 의 pressed → EventManager.advance_line() .

run_ui.gd::screens 에 "event": \$event_screen 추가. show_phase("event") 시 자동 노출.

검증 (실 플레이): 노드 진입 → 화면 자동. 라인 진행 → 마지막 → 화면 사라지고 phase 복귀.

§6. 통합 검증 시나리오 (MVP)

1. **dialogue 단일** — 라인 N개 진행 → 자동 종료. `seen_events[id] = 1`, `phase = "map"`.
2. **once_per "big_run"** — 회귀 후 같은 풀에서 제외 (`_resolve_event_for_node` 가 필터).
3. **이벤트 노드 + 일반 적 노드 혼재** — type 분기 정상 동작 검증.

(다음 차로 시나리오) 4. **effect 단일** — UI 입력 없이 즉시 통과. `_apply_*` 호출 검증. 5. **chain (dialogue → effect → dialogue)** — phase 안 빠지고 chain 자동 진행. `seen_events` 는 chain root 만 카운트. 6. **choice 분기** — 선택 시 가지의 `next` 로 전이.

§7. 자가점검

삼항 — 읽기 테스트: `event_manager.gd` + `run_manager.gd` + `game_data.gd` 만으로 (1) 정의 위치 (2) 트리거 평가 (3) kind 디스패치 (4) 발생 이력 모두 답 가능. UI 코드 의존 0.

삼항 — 교체/삭제 테스트: `event_ui.gd` 삭제 시 시물 컴파일 에러 0건. 콘솔만으로 동일 작동.

자료구조 사이클:

자료구조	위치	생성	변경	조회
EventDefinition	game_data	상수	N/A	EventManager
event_state	EventManager	begin_event	advance_line	event_ui
seen_events	big_run_data	_new_big_run	event_resolved 핸들러	EventManager (필터)
phase "event"	run_data	begin_event 진입 시	event_resolved 시 복귀	run_ui, event_ui

코어 원칙 검증: - RunManager 가 영속 상태 단일 출처 ✓ (seen_events / phase) - EventManager 는 일시 상태 만 (battle_state 와 동일 격) ✓ - Type 1 액션 적용 단일 경로 = `_apply_*` ✓ (effect kind 도입 시점 검증) - 새 매 니저 분리 정당화: lifecycle (`begin_event ~ event_resolved`) + discrete structure (`event_state`) — BattleManager 기준 충족.

§8. 커밋 단위 (MVP — 2건)

1. **EventManager 오토로드 + dialogue kind 흐름** — `event_manager.gd` 신설, `EVENTS` 상수, 노드 type `"event"`, `begin_event` / `advance_line` / `event_resolved`, RunManager 분기 + 핸들러, 콘솔 명령
 2. **event_ui.gd + main.tscn 배선** — 라인 렌더, advance 입력
-

§9. 다음 차로 (스키마 자리는 마련되어 있음)

1. **effect kind** 디스패처 — `_dispatch_effect` 가 `effects` 배열 순회하며 `RunManager._apply_*` 호출 → 즉시 `_finalize_event` . UI 통과.
 2. **next 필드 + chain runner** — `_finalize_event` 직전에 `next` 검사. 있으면 `begin_event(next)` 호출 (phase 유지). `chain_root_id` 가 `seen_events` 의 카운트 키.
 3. **movie kind** — `VideoStreamPlayer` 통합. `skippable` 옵션. 재생 종료 시그널 → `_finalize_event` .
 4. **choice kind** — UI 가 `prompt` + 버튼 `choices` 렌더. 선택 시 `_finalize_event` 가 그 가지의 `next` 로 전이.
 5. **CHARACTERS 레지스트리** — `speaker` 키 → `display_name` / `color` / `portraits` lookup.
 6. 변수 인터플레이션 — `text` 의 `{run.body_hp}` 등 토큰 치환.
 7. 외부 **.json** 파일 분리 — 이벤트 수가 임계점 넘은 후. `const dict` → `JSON.parse_string` .
 8. **once_per: "internal_run"** — 내부 런 단위 카운트.
 9. **트리거 확장** — `combat_victory` / `recurrence_count` / 합성.
- §0 의 8 항 (A~H) 확정 후 코드 진입.